

Transcrição do Material interativo: Processos Pesados

Página 1

Já vimos na seção introdutória e no Módulo 1 o conceito de Processos.

Página 2

Bom, sendo um Processo um programa em execução, algumas informações referentes ao contexto dessa execução são organizadas em contador de programa, pilha e seção de dados.

Página 3

Na memória, temos a representação dada por esta figura. Então, partindo do endereço zero do nosso exemplo, temos a seção de texto, que nada mais é do que o código em execução; é o conjunto de instruções que devem ser executadas uma a uma para obter o resultado final esperado pelo programa em caso de sucesso.

A seção de dados guarda informações sobre as variáveis globais do programa, o *heap*, as informações de alocação e a dinâmica de memória. Existem situações em que o programa necessita de mais memória e, no momento da execução, ele faz essa solicitação ao sistema operacional. Como vocês veem, há uma seta apontada para cima. Isso indica que essa seção tem tamanho variável, ou seja, ela pode ir aumentando seu tamanho, não ilimitadamente. Lembrem-se de que a memória é um recurso finito.

E por último, a pilha. Ela guarda informações sobre variáveis locais, chamadas e retorno de funções.

Página 4

Os estados de um processo podem seguir de acordo com essa visão simplificada. Um processo pode:

- Novo: quando ele está sendo criado.
- Executando: quando tem suas instruções sendo executadas.
- Esperando: quando o processo está esperando algum evento acontecer.
- Apto: o processo está esperando ser associado a um processador para ser executado.
- Terminado: quando o processo terminou a sua execução.

Página 5

Vocês já conhecem esses estados. Para recapitularmos, percebemos o estado em que se considera a suspensão dos processos, que ocorre exatamente quando um processo ativo no sistema tem o seu conteúdo movido da memória principal para a memória secundária como o disco, por exemplo. E por que é importante mencionar esses dois estados? Porque sempre que o processo faz uso do processador, ele precisa estar carregado na memória, principalmente nesse caso, um processo suspenso deve ser recolocado em memória.

Página 6

Conhecendo, então, os estados do processo em execução e sabendo que eles podem ser *preemptados*, ou seja, ter sua execução interrompida por um tempo, o bloco de controle do processo é a estrutura utilizada para armazenar informações sobre o *status* do processo em execução. Neste caso, temos o estado do processo, que vimos anteriormente; o contador de programas, que aponta a próxima instrução a ser executada; o conteúdo dos registradores do processador; informações sobre como os programas são selecionados; a ordem para serem executados; a região de memória utilizada; a quantidade e o percentual de uso dos recursos; e, ainda, informações sobre os arquivos que possam estar manipulando. Além disso, temos o identificador do processo que deve ser o único no sistema.

Página 7

Então, quando todos os programas têm sua seção interrompida, seja por alguma interrupção de Hardware ou Software chamada de sistema, seja por esgotamento do *timer* por exemplo, o sistema precisa trocar de contexto, ou seja, o estado atual do processo precisa ser salvo e o contexto desse processo é representado pelo PCB. É importante ressaltar que há um custo associado a essa troca de contexto. Neste caso, um sistema que faz muita troca de contexto não é muito eficiente, já que não é um trabalho útil que está sendo realizado. Não é a execução de um processo, por exemplo.

Página 8

Aqui temos um exemplo de troca de contexto. O processo zero está em execução e em dado instante de tempo é interrompido. O sistema operacional deve salvar o contexto do processo para atender a essa interrupção. Nesse exemplo, o processo fica bloqueado esperando o atendimento de uma requisição de I/O, por exemplo. E nesse mesmo momento, o processo 1 é retirado da fila de aptos para a execução no processador, onde carrega-se as informações de seu PCB no sistema. A requisição de I/O foi atendida, agora o processo 1 é *preemptado* e o processo zero volta para execução.

Página 9

Vejam que existem diferentes filas mantidas pelo sistema operacional: fila de processos aptos que guardam uso do processador; fila de *jobs*, que são todos os processos do sistema; e também a fila para os dispositivos, já que mais de um processo pode tentar acessar o dispositivo ao mesmo tempo. Além disso, os processos podem migrar de uma fila para outra, dependendo da sua execução no sistema. O próprio PCB é utilizado como conteúdo das filas.

Página 10

Como vimos no início do módulo, aqui estão as diferentes filas do sistema. Percebam que em cada fila o PCB do início aponta para o próximo, e assim segue para o final da fila. Além disso, mantém as informações de quem é o PCB do início e do final da fila, justamente para deixar mais eficiente as operações de inserção e remoção.

Página 11

Sabendo dessas informações, eu pergunto a vocês: como os processos são criados? Há uma hierarquia entre os processos existentes, uma relação de pai e filho, por exemplo?

Cada processo criado no sistema tem um identificador único e, além disso, pais e filhos podem compartilhar recursos e arquivos, por exemplo, ou podem também não compartilhar. Pais e filhos executam de modo concorrente, e o processo pai espera o término da execução dos processos filhos. Imaginem a seguinte situação: vocês abrem o terminal no Linux e digitam o comando LS. LS é um programa, portanto, ele é filho do *bech*, que foi o programa executado quando o terminal foi aberto. Se fecharmos o terminal enquanto um programa estiver executando, estaremos interrompendo sua execução. No Linux, todos os processos são filhos do *unit*.

Página 12

Como os processos são criados no sistema UNIX? Ao digitar LS no nosso exemplo, foi feita uma chamada de sistema, nomeada de *fork*. Neste caso, o sistema sabe que um novo processo deve ser criado. Ao criar o processo filho, o espaço do pai é duplicado como LS e esse é um programa diferente, em que precisamos substituir a seção de código e as demais informações do processo. Nesse momento, é executada a chamada de sistema *exec*, que substitui o código do processo a ser executado.

Página 13

Então, temos aqui um exemplo das chamadas de sistema *fork* e *exec*, na qual o *fork* é utilizado para criar o processo filho, e o *exec* para substituir o código do processo pai. Vemos a relação de parentesco, em que o pai espera pelo término da execução do processo filho.

Página 14

Aqui temos um exemplo em C usando *fork* e *exec* também. *Fork* é uma chamada de sistema que retorna um valor menor que 0 quando ocorre algum erro e o filho não pode ser criado. Ele retorna o valor 0 para o processo filho, e retorna o ID do processo criado para o pai.

Página 15

E como são encerrados os processos? O processo que executa *exit* terminou normalmente. Se ele aguarda a finalização do processo filho via *wait*, os dados de saída passam do filho para o pai.